
Process Management

Process Alternative - Threads

Lecture 4 (**Live** Version)

Overview

■ **Threads:**

- ❑ Motivation
- ❑ Basic Idea

■ **Threads models:**

- ❑ Kernel vs User Thread
- ❑ Hybrid model

■ **Thread in Unix**

- ❑ POSIX thread:
 - Create, Exit, Synchronization
 - Exploration

Motivation for Thread

- Process is **expensive**:
 - **Process creation** in the `fork()` model:
 - Duplicate memory space & process context
 - **Context switch**:
 - Requires saving/restoration of process Information
- Communication between processes:
 - Independent memory space → No easy way to pass information
 - Requires **Inter-Process Communication (IPC)**

Motivation for Thread (cont)

- **Thread** was invented to overcome the problems with process model
 - Started out as a "*quick hack*"
- **Basic Idea:**
 - Traditional process has a **single thread of control**:
 - Only one instruction of the whole program is executing at any time
 - Add **more threads of control** to the same process:
 - Multiple parts of the programs is executing at the same time conceptually

Threads of control: Illustration

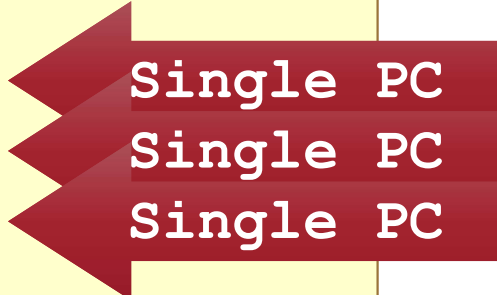
LUNCH TIME!!

```
int main()  
{  
    steamRice( twoBowls );  
    fryFish( bigFish );  
    cookSoup( cornSoup );  
  
    printf( "Lunch READY!!\n" );  
    return 0;  
}
```

Threads of control: Illustration (cont)

- Single Threaded Process go through the functions **sequentially**

```
int main()  
{  
    steamRice( twoBowls );  
    fryFish( bigFish );  
    cookSoup( cornSoup );  
  
    printf( "Lunch READY!!\n" );  
    return 0;  
}
```



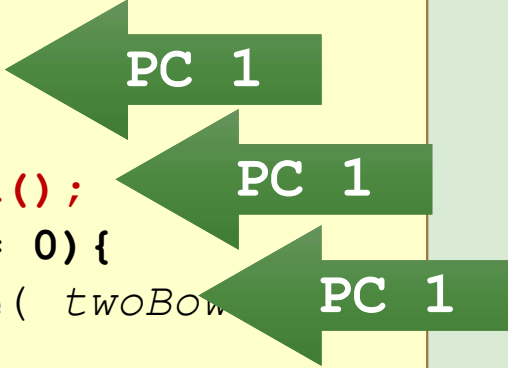
Single PC
Single PC
Single PC

Time

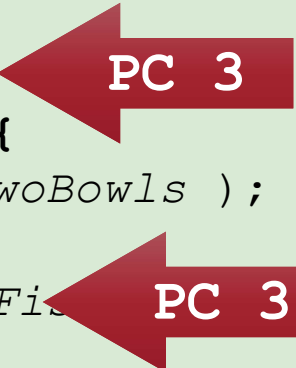


Multiple "threads of control"

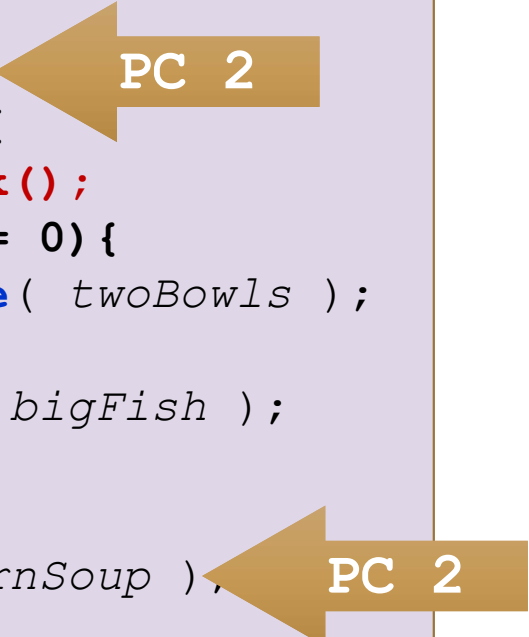
```
int result;  
result = fork();  
if (result != 0){  
    result = fork();  
    if (result != 0){  
        steamRice( twoBowls );  
    } else {  
        fryFish( bigFish );  
    }  
} else {  
    cookSoup( cornSoup );  
}  
... ..
```



```
int result;  
result = fork();  
if (result != 0){  
    result = fork();  
    if (result != 0){  
        steamRice( twoBowls );  
    } else {  
        fryFish( bigFish );  
    }  
} else {  
    cookSoup( cornSoup );  
}  
... ..
```



```
int result;  
result = fork();  
if (result != 0){  
    result = fork();  
    if (result != 0){  
        steamRice( twoBowls );  
    } else {  
        fryFish( bigFish );  
    }  
} else {  
    cookSoup( cornSoup );  
}  
... ..
```



We effectively
have two
"threads" of
control after
fork()

New approach: **Multi-Threading**

```
int main()
```

```
{
```

```
  Thread 1 do  steamRice( twoBowls );
```

```
  Thread 2 do  fryFish( bigFish );
```

```
  Thread 3 do  cookSoup( cornSoup );
```

```
    Wait for all threads to finish;
```

```
    printf( "Lunch Ready\n" );
```

```
    return 0;
```

```
}
```

PC 1

PC 2

PC 3

Process and Thread

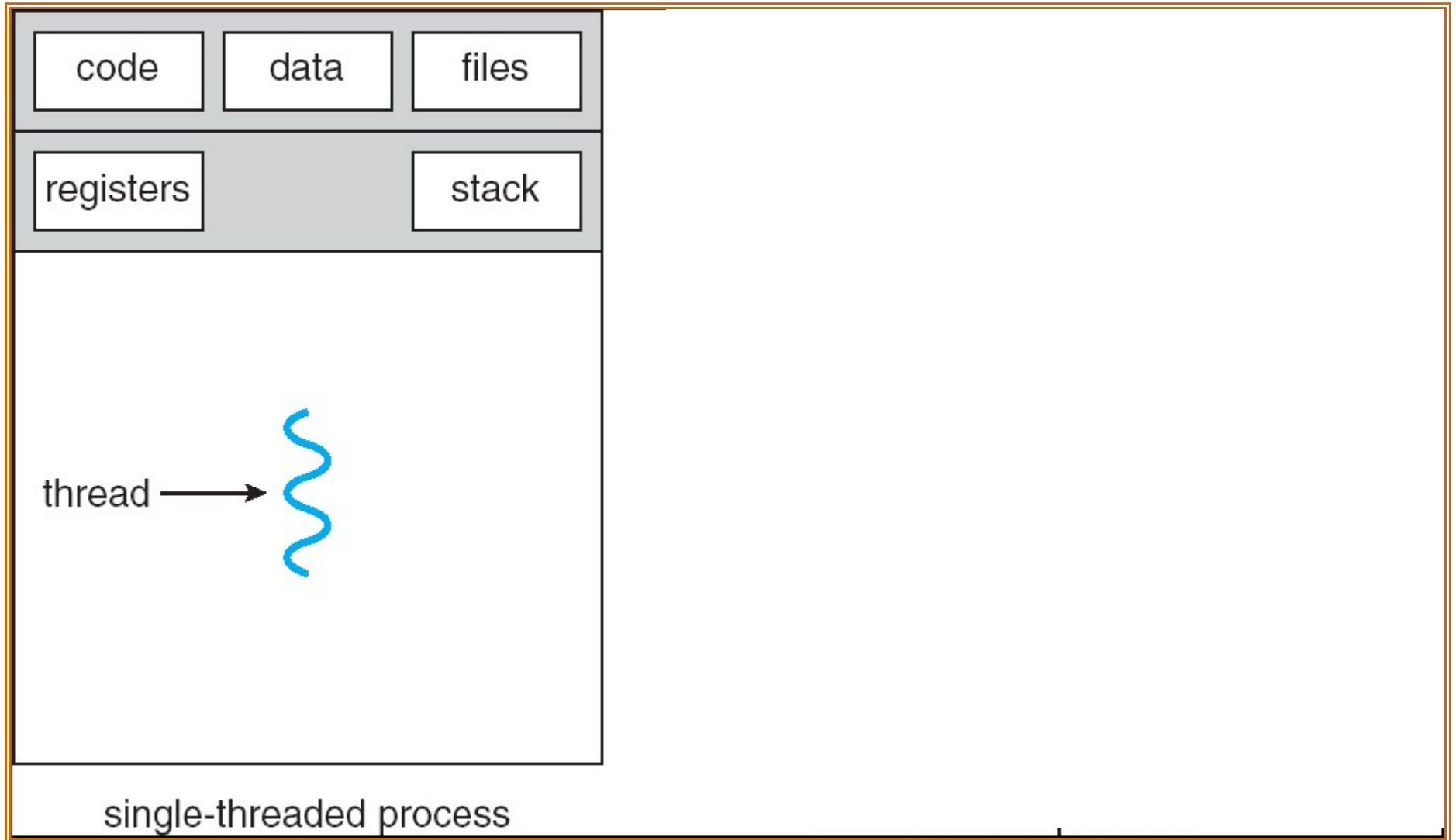
- A single process can have multiple threads:
 - Known as **multithreaded process**

Quick Check:

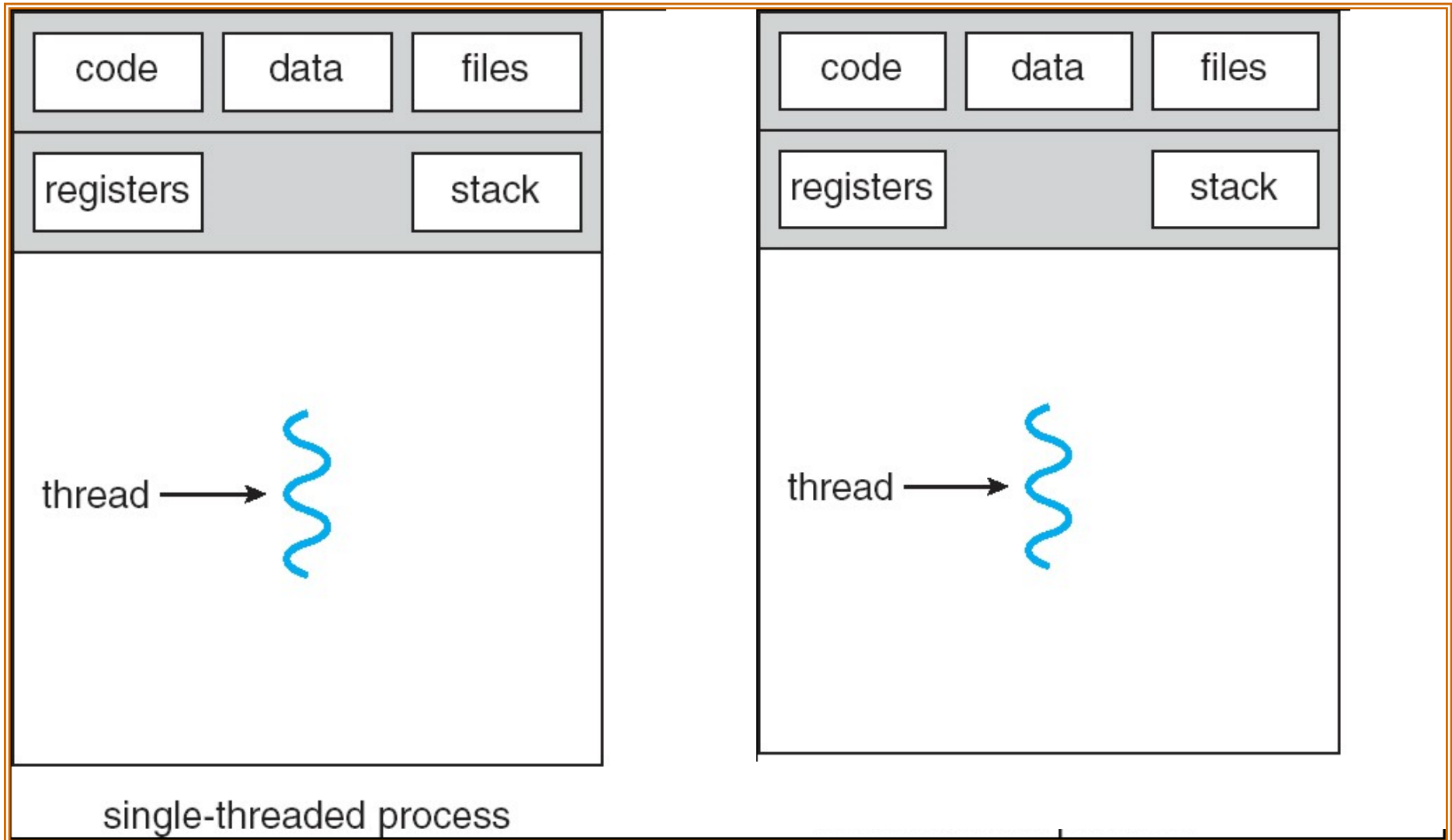
What can we share between threads?

- Unique information needed for each thread:
 - Identification (usually **thread id**)
 - Registers (General purpose and special)
 - “Stack” (more about this later)

Process and Thread: Illustration



Process Context Switch VS Thread Switch



Threads: **Benefits**

Economy:

- Multiple threads in the same process requires much less resources to manage compared to multiple processes

Resource sharing:

- Threads share most of the resources of a process
- No need for additional mechanism for passing information around

Responsiveness:

- Multithreaded programs can appear much more responsive

Scalability:

- Multithreaded program can take advantage of multiple CPUs

Thread: **Problems**

■ **System Call Concurrency**

- Parallel execution of multiple threads → parallel system call possible

■ **Process Behavior:**

- Impact on process operations
- Example:
 - `fork()` duplicate process, how about threads?
 - If a single thread executes `exit()`, how about the whole process?
 - If a single thread calls `exec()`, how about other threads?

Thread Models

What are the ways to support threads?

Two Major Thread Implementations

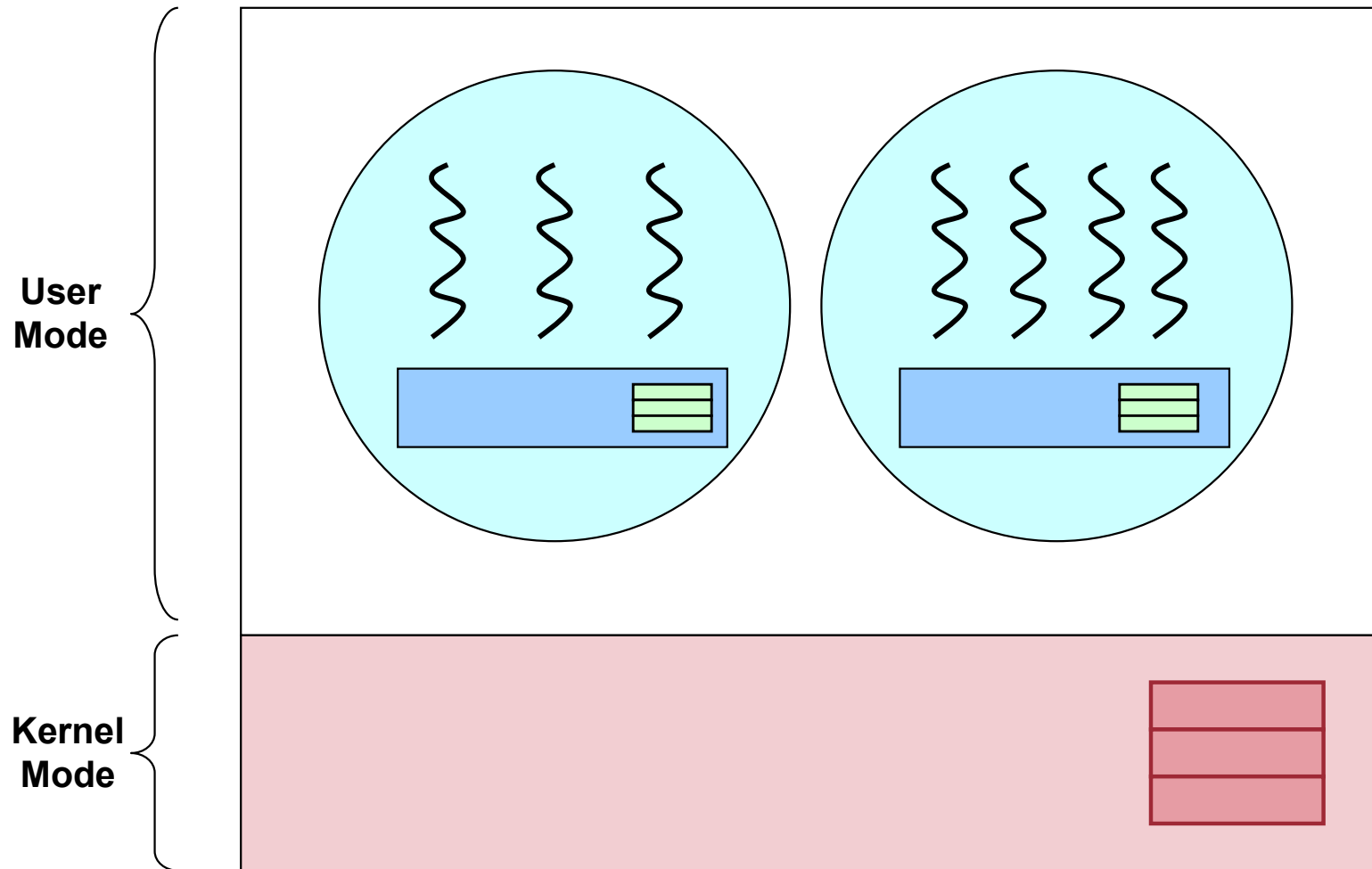
User Thread:

- Thread is implemented as a **user library**
 - A **runtime system** (in the process) will handle thread related operation
- Kernel is **not aware** of the threads in the process

Kernel Thread:

- Thread is implemented in the **OS**
 - Thread operation is handled as **system calls**
- Thread-level scheduling is possible:
 - Kernel schedule by threads, instead of by process
- Kernel may make use of threads for its own execution

User Thread: Illustration



User Thread: **Pros** and **Cons**

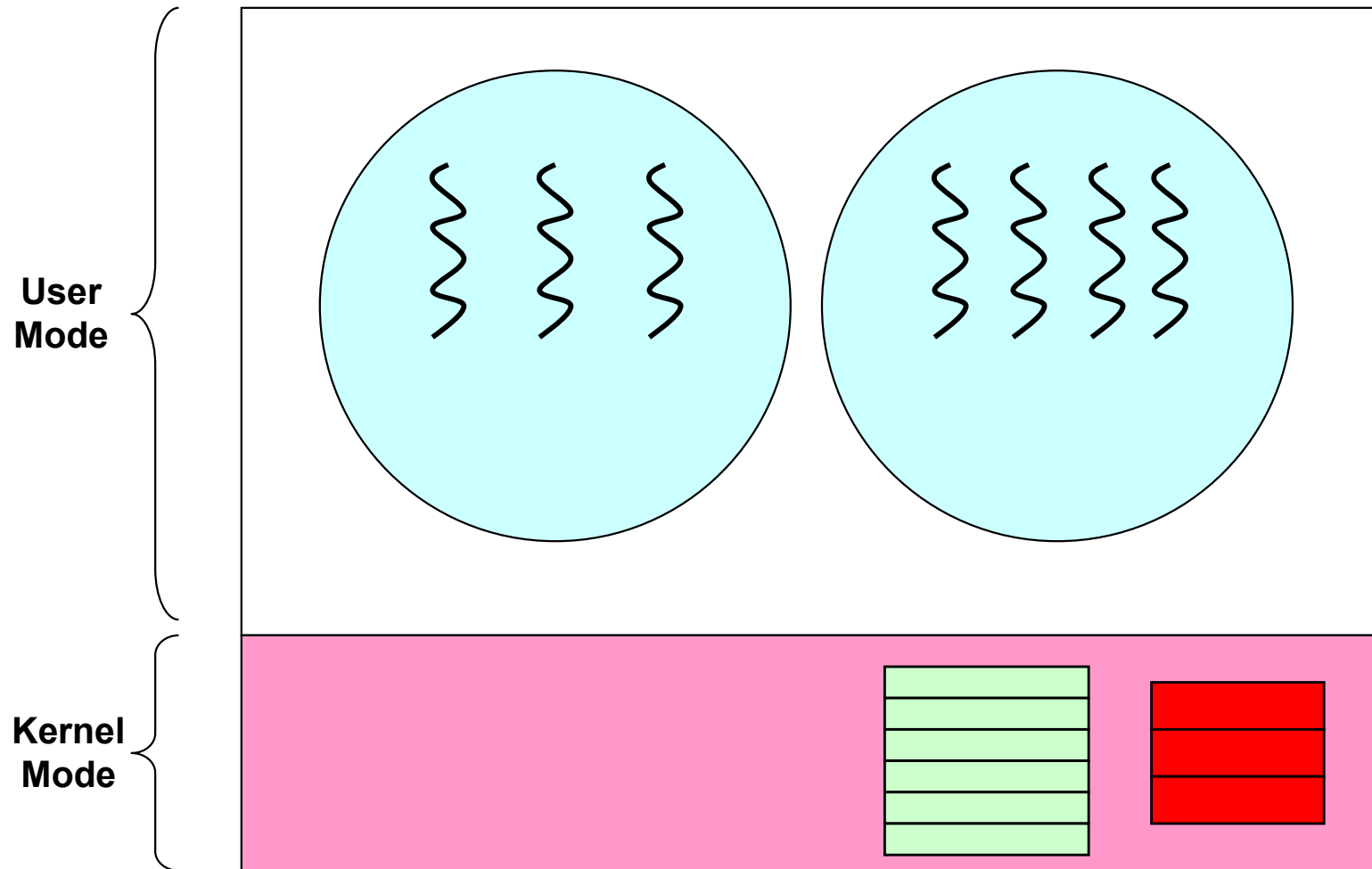
■ **Advantages:**

- ❑ Can have multithreaded program on ANY OS
- ❑ Thread operations are just library calls
- ❑ Generally more configurable and flexible
 - e.g. Customized thread scheduling policy

■ **Disadvantages:**

- ❑ OS is not aware of threads, scheduling is performed at process level
 - One thread blocked → Process blocked → All threads blocked
 - **Cannot exploit multiple CPUs!**

Kernel Thread: Illustration



Kernel Threads: **Pros** and **Cons**

■ **Advantages:**

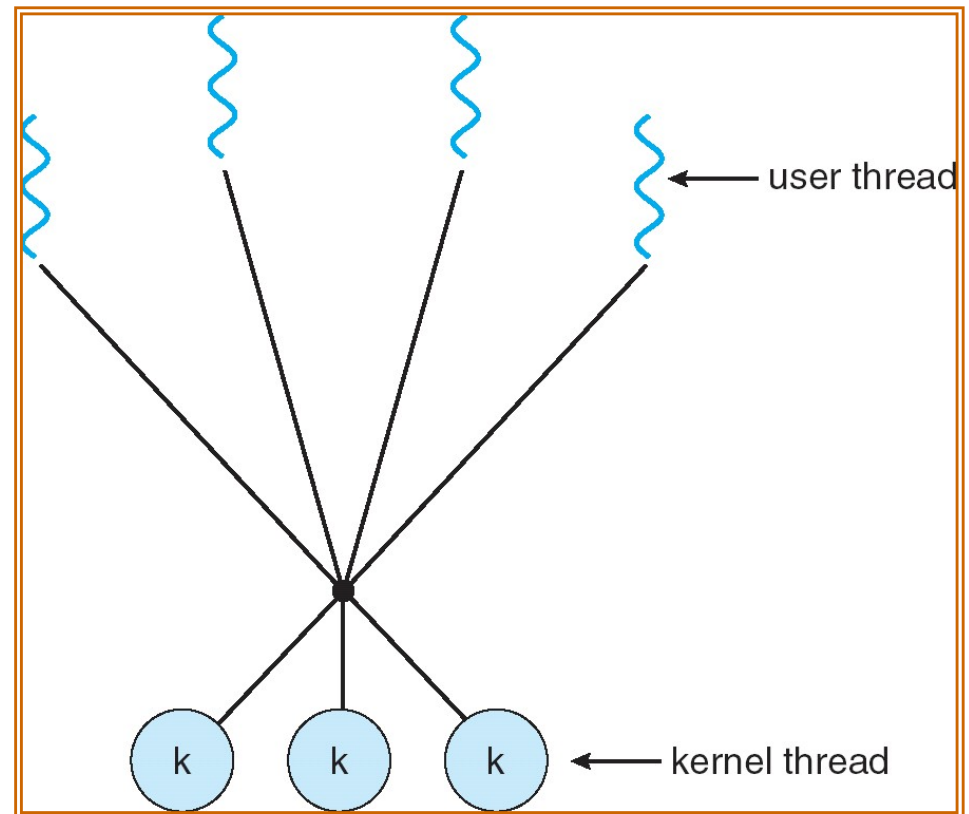
- ❑ Kernel can schedule on thread levels:
 - More than 1 thread in the same process can run simultaneously on multiple CPUs

■ **Disadvantages:**

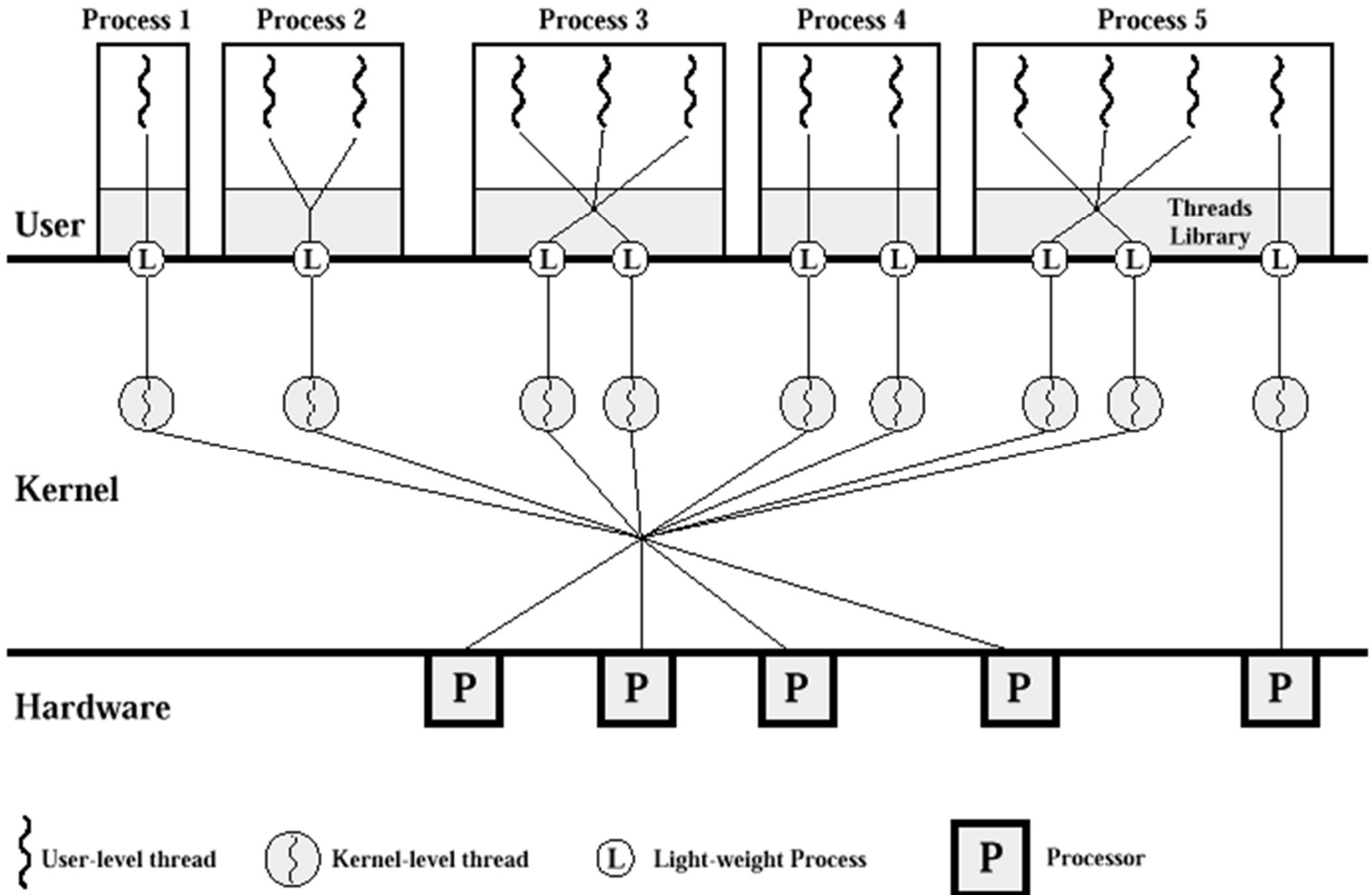
- ❑ Thread operations is now a system call!
 - Slower and more resource intensive
- ❑ Generally less flexible:
 - Used by all multithreaded programs
 - If implemented with many features:
 - ❑ Expensive, overkill for simple program
 - If implemented with few features:
 - ❑ Not flexible enough for some programs

Hybrid Thread Model

- Have both **Kernel** and **User** threads
 - ❑ OS schedule on **kernel threads** only
 - ❑ **User thread** can binds to a **kernel thread**
- Offer great flexibility
 - ❑ Can limit the concurrency of any process / user



Hybrid Model Example: **Solaris**



Threads on Modern Processor

		Brand Name & Processor Number ¹	Base Clock Speed (GHz)	Turbo Frequency ² (GHz)	Cores/Threads
■ T	■	NEW Intel® Core™ i7 4960X Unlocked	3.6	Up to 4.0	6/12
■ T	■	NEW Intel® Core™ i7 4930K Unlocked	3.4	Up to 3.9	6/12
■ T	■	NEW Intel® Core™ i7 4820K Unlocked	3.7	Up to 3.9	4/8
■ T	■	Intel® Core™ i7-4770K Unlocked	3.5	Up to 3.9	4/8

intel inside CORE™ i7

POSIX Threads

An example thread API

POSIX Threads: *pthread*

- Standard defined by the IEEE
 - Supported by most Unix variants
- Defines the API as well as the behavior
 - However, **implementation** is not specified
 - So, *pthread* can be implemented as user / kernel thread
- Will show a few example to highlight the differences between process and thread only

Basics of Pthread

- Header File:

- ❑ `#include <pthread.h>`

- Compilation:

- ❑ `gcc XXXX.c -lpthread`

- Useful datatypes:

- ❑ `pthread_t`: Data type to represent a thread id (TID)

- ❑ `pthread_attr_t`: Data type to represents attributes of a thread

Pthread Creation

■ Thread creation syntax:

```
int pthread_create(  
    pthread_t    *tidCreated,  
    const pthread_attr_t *threadAttributes,  
    void*  (*startRoutine) (void*),  
    void *argForStartRoutine );
```

■ Returns:

- ❑ 0 to indicate success; !=0 for errors

■ Parameters:

- ❑ **tidCreated**: Thread Id for the created thread
- ❑ **threadAttributes**: Control the behavior of the new thread
- ❑ **startRoutine**: Function pointer to the function to be executed by thread
- ❑ **argForStartRoutine**: Arguments for the `startRoutine` function

Pthread Termination

- Thread termination syntax:

```
void pthread_exit( void* exitValue );
```

- Parameters:

- `exitValue`: Value to be returned to whoever synchronize with this thread (more later)

- If `pthread_exit()` is not used, a *pthread* will terminate automatically when the end of the `startRoutine` is reached

- In this case, there is no way to return a exit value

Pthread Creation & Termination: Example

```
//header files not shown

void* sayHello( void* arg )
{
    printf("Just to say hello!\n");
    pthread_exit( NULL );
}

int main()
{
    pthread_t tid;

    pthread_create( &tid, NULL, sayHello, NULL );
    printf("Thread created with tid %i\n", tid);

    return 0;
}
```

Pthread: Sharing of memory space

```
//header files not shown
int globalVar;

void* doSum( void* arg)
{
    int i;
    for (i = 0; i < 1000; i++)
        globalVar++;
}

int main()
{
    pthread_t tid[5];    //5 threads id
    int i;
    for (i = 0; i < 5; i++)
        pthread_create( &tid[i], NULL, doSum, NULL );
    printf("Global variable is %i\n", globalVar);
    return 0;
}
```

- What is the sum that we get (or should get)?

Pthread: Simple Synchronization

- To wait for the termination of another pthread:

```
int pthread_join ( pthread_t threadID,  
                  void ** status );
```

- Returns:

- 0 to indicate success; != 0 for errors

- Parameters:

- threadID: **TID** of the pthread to wait for
- status: Exit value returned by the target pthread

Pthread: Sharing of memory space V2.0

```
//header files not shown
int globalVar;

void* doSum( void* arg)
{ //same as before }

int main()
    pthread_t tid[5];    //5 threads id
    int i;
    for (i = 0; i < 5; i++)
        pthread_create( &tid[i], NULL, doSum, NULL );

    //Wait for all threads to finish
    for (i = 0; i < 5; i++)
        pthread_join( tid[i], NULL );

    printf("Global variable is %i\n", globalVar);
    return 0;
}
```

Pthread: A lot more!

- There are more interesting stuff about ***pthread***:
 - ❑ Yielding (giving up CPU voluntarily)
 - ❑ Advanced synchronization
 - ❑ Scheduling policies
 - ❑ Binding to kernel threads
 - ❑ Etc
- As we cover new topics, you can explore the ***pthread*** library to see the application!

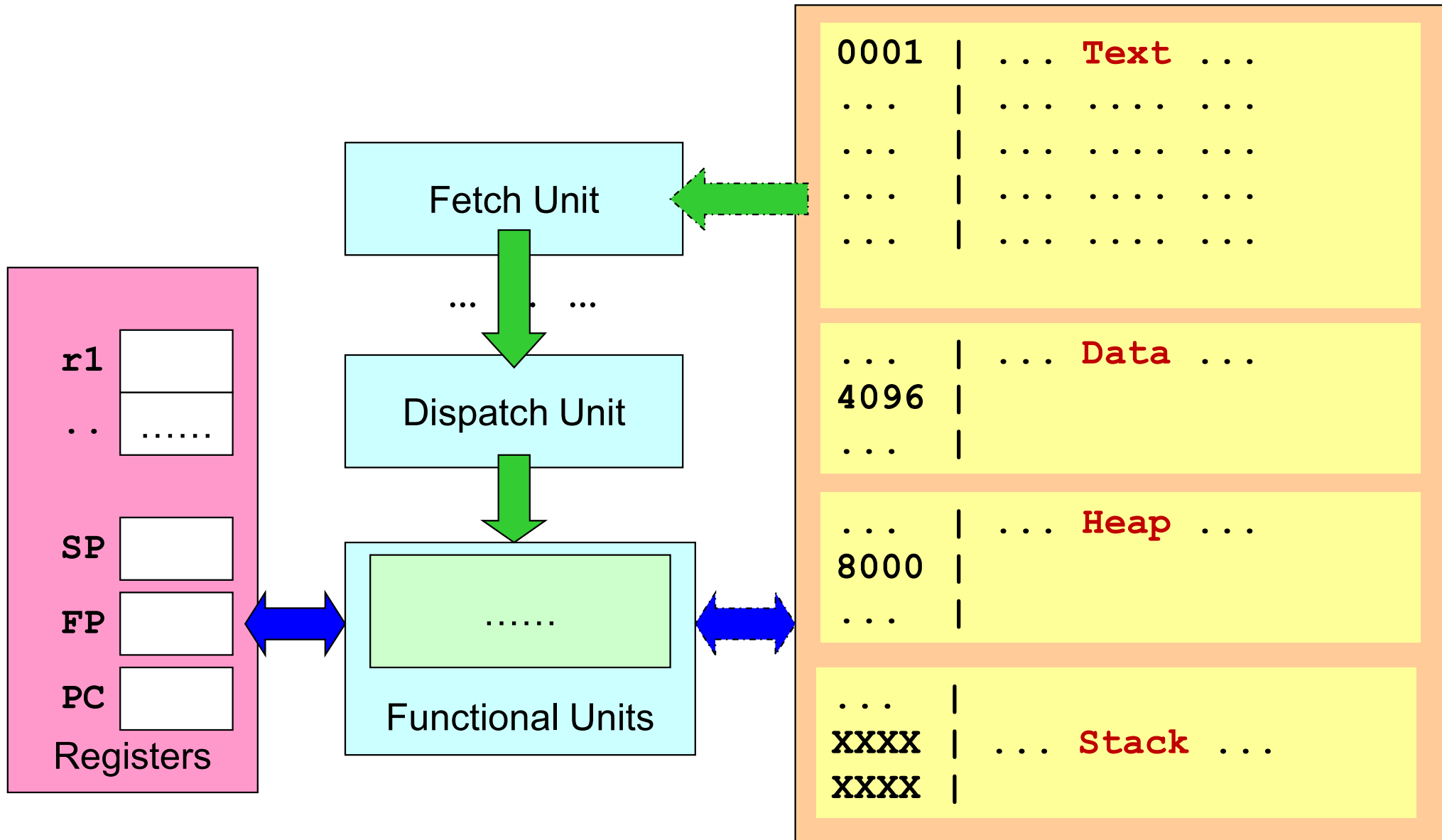
Summary

- Thread:
 - ▣ Motivation
 - ▣ Difference between thread and process
- Thread Models:
 - ▣ User vs Kernel thread
- Simple introduction to pthread library

Reference

- Modern Operating System (3rd Edition)
 - ▣ Chapter 2.2
- Operating System Concepts (7th Edition)
 - ▣ Chapter 4
- Linux Pthread Man Pages
 - ▣ “man pthread.....” for more

What can we **share**?





Disclaimer: I cheated a little 😊

- Accessing shared variable is **much more trickier** than shown!
- The "`globalVar++`" is NOT a single machine operation:
 1. Read `globalVar` from memory
 2. Increase `globalVar`
 3. Store `globalVar` back to memory
- What if a thread is switched out in step 2?
- So, a small number of iterations is chosen to avoid thread switching in middle 😊

Process Context Switch VS Thread Switch

- **Process context switch** involves:
 - ❑ OS Context
 - ❑ Hardware Context
 - ❑ Memory Context
- **Thread switch** within the same process involves:
 - ❑ Hardware context:
 - Registers
 - "Stack" (actually just changing FP and SP registers)
- Thread is much “lighter” than process:
 - ❑ a.k.a. ***lightweight process***